Ishaan Bhardwaj, A2305223339

PRACTICAL 1

AIM: To study about UML and Rational Rose Environment

THEORY:

Unified Modeling Language (UML) is a standardized modeling language used in software engineering to visually represent the design and architecture of a system. It helps developers and stakeholders understand, analyze, and document software systems effectively before implementation.

Key features of UML:

- UML is **platform-independent** and supports object-oriented concepts.
- It provides a **common visual language** for communication among developers, analysts, and clients.
- UML improves **system clarity**, reduces design errors, and enhances maintainability.
- It can be used throughout the **Software Development Life Cycle (SDLC)**.

Types of UML diagrams:

- **Structural Diagrams**
  - Class Diagram
  - Object Diagram
  - Component Diagram
  - Deployment Diagram
- **Behavioral Diagrams**
  - Use Case Diagram
  - Sequence Diagram
  - Activity Diagram
  - State Machine Diagram

These diagrams together represent both the **static structure** and **dynamic behavior** of a system.

The **Rational Rose Environment** is a widely used UML-based software modeling tool developed by IBM. It provides a graphical user interface to design and manage UML diagrams efficiently.

Key features of Rational Rose:

- Supports creation and modification of all standard UML diagrams.
- Enables **round-trip engineering**, allowing synchronization between UML models and source code.
- Supports object-oriented languages such as **Java, C++, and Visual Basic**.
- Provides **model validation** to ensure design consistency.
- Facilitates **documentation generation** and version control integration.

Advantages of using UML with Rational Rose:

- Improves design accuracy and reduces development risks.
- Enhances collaboration among team members.
- Helps in early detection of design flaws.
- Ensures systematic and organized software development.

Ishaan Bhardwaj, A2305223339

PRACTICAL 2

AIM: To study and design Class Diagram in UML for Book Bank System

THEORY:

A **Class Diagram** is one of the most important **structural diagrams** in UML. It represents the static structure of a system by showing its classes, attributes, methods, and the relationships among classes. Class diagrams are mainly used during the design phase of software development to model object-oriented systems clearly. They help developers understand how different components of a system are organized and how they interact with each other at a high level.

Notations Used in Class Diagram:

- **Class**: Represented by a rectangle divided into three sections:
  1. Class name
  2. Attributes (data members)
  3. Methods (member functions)
- **Association**: A straight line showing a relationship between two classes.
- **Multiplicity**: Indicates the number of objects involved in a relationship (e.g., 1, 0.., *1*..).
- **Generalization (Inheritance)**: Represented by a solid line with a hollow arrow pointing to the parent class.
- **Aggregation**: Shown using a hollow diamond, representing a “has-a” relationship.
- **Composition**: Shown using a filled diamond, indicating a strong ownership relationship.
- **Visibility**:
  1. + Public
  2. - Private
  3. # Protected

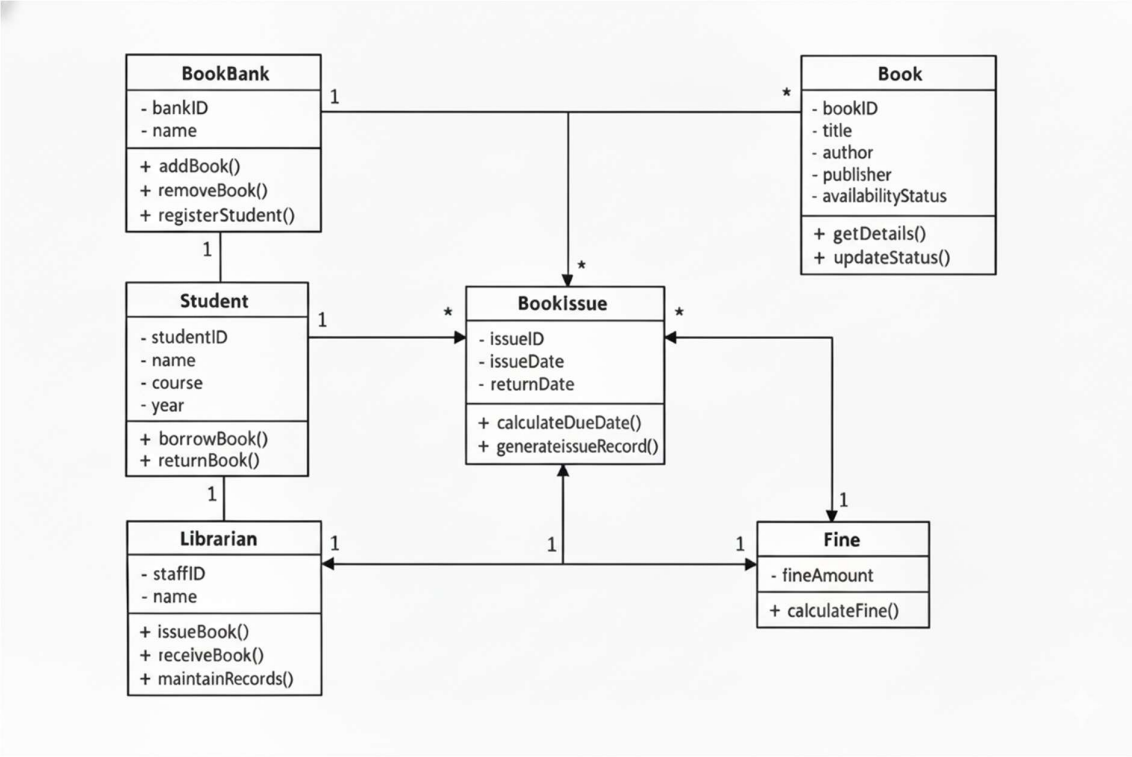
Ishaan Bhardwaj, A2305223339

A **Book Bank System** is an automated system used in educational institutions to manage the issue, return, and record of books provided to students, usually for an academic term.

Classes in Book Bank System UML

- **Book**: Stores details such as book ID, title, author, publisher, and availability status.
- **Student**: Maintains student information like student ID, name, course, and year.
- **Librarian**: Handles book issuing, returning, and record maintenance.
- **BookIssue**: Manages issue date, return date, and fine details.
- **BookBank**: Acts as the central class to manage books and users.
- **Fine**: Calculates penalties for late return of books.

UML DIAGRAM:



Ishaan Bhardwaj, A2305223339

PRACTICAL 3

AIM: To study and design Use Case Diagram in UML for Book Bank System

THEORY:

A **Use Case Diagram** is a type of **behavioral diagram** in the Unified Modeling Language (UML) that represents the functional requirements of a system. It shows how different **actors** (users or external systems) interact with the system to achieve specific goals. Use case diagrams focus on **what the system does**, rather than how it is internally implemented, making them easy to understand for both technical and non-technical stakeholders.

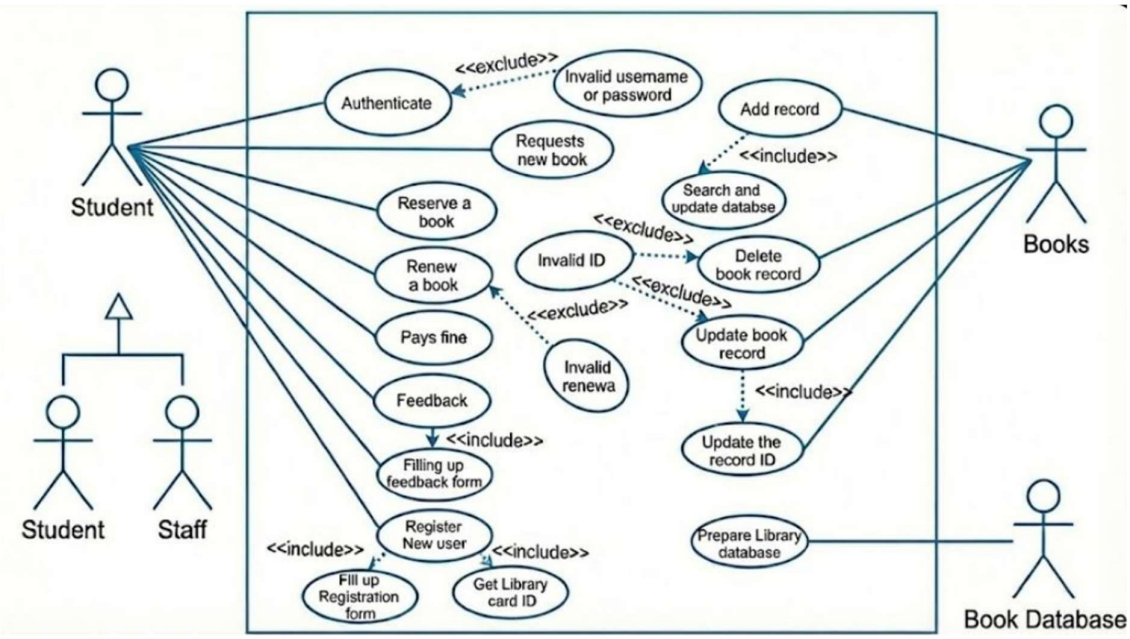
Use case diagrams are mainly used during the **requirement analysis phase** of the Software Development Life Cycle (SDLC). They help in identifying system functionalities, defining system boundaries, and improving communication between users, developers, and clients. Each use case represents a specific service or action provided by the system.

Notations Used in Use Case Diagram

- **Actor**: Represented by a stick figure; denotes a user or external entity interacting with the system.
- **Use Case**: Represented by an oval; shows a specific function performed by the system.
- **System Boundary**: A rectangle that encloses all use cases of the system.
- **Association**: A line connecting an actor to a use case, indicating interaction.
- **Include Relationship**: Shows mandatory behavior reused by another use case.
- **Extend Relationship**: Represents optional or conditional behavior.

Ishaan Bhardwaj, A2305223339

USE CASE DIAGRAM:



PRACTICAL 4

AIM: To study the concept of State Diagram in UML and to design a UML State Diagram that represents the different states of an order in an E-commerce Order Processing System, from order creation to completion or cancellation.

THEORY:

A **State Diagram** in UML (also called a **State Machine Diagram**) is used to model the **dynamic behavior** of a system by showing how an object changes its **state** in response to **events** over time. It focuses on the lifecycle of an object and the different states it can occupy during its existence.

State diagrams are especially useful for systems where the behavior depends on the current state, such as **banking systems, library systems, traffic lights, vending machines**, etc.

The main purposes of a UML state diagram are:

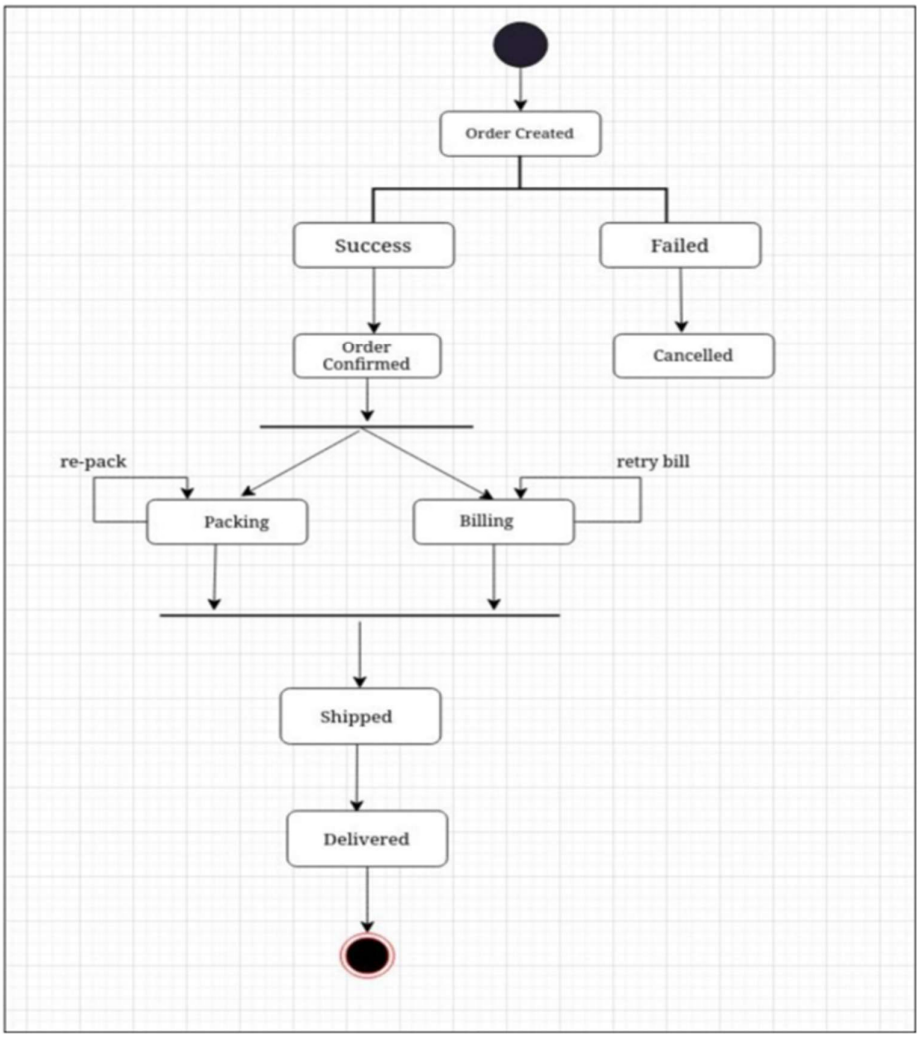
- To represent the **states of an object**
- To show **state transitions** triggered by events
- To model **reactive and event-driven systems**
- To understand and visualize the **lifecycle of an object**
- To help in **system analysis and design**

Notations in UML State Diagram

- **State**: Represents a condition of an object; shown by a **rounded rectangle**.
- **Initial State:** Starting point of the state diagram; shown by a **solid black circle**.
- **Final State:** End of the state diagram; shown by a **double circle**.
- **Transition:** Movement from one state to another; shown by a **directed arrow**.
- **Event:** An occurrence that triggers a transition.
- **Guard Condition:** A Boolean condition that must be true for transition; written in **[ ]**.
- **Action:** Operation performed during a transition; written after /.
- **Activity:** Operation performed while staying in a state.
- **Entry Action:** Action executed when entering a state.
- **Exit Action:** Action executed when leaving a state.

- **Self-Transition:** Transition where a state moves back to itself.
- **Composite State:** A state containing sub-states.

STATE DIAGRAM:



Ishaan Bhardwaj, A2305223339

PRACTICAL 5

AIM: To study Object Diagrams in UML. A banking system consists of customers and bank accounts. Each customer can have multiple accounts. At a particular moment, customer Emma owns two accounts: a savings account with a balance of $10,000 and a checking account with a balance of $2,500. Draw an object diagram representing this situation.

THEORY:

An Object Diagram is a type of structural diagram in the Unified Modeling Language (UML) that represents a snapshot of a system at a particular moment in time. Unlike a class diagram, which shows the blueprint of the system, an object diagram illustrates actual instances (objects) of classes along with their current attribute values and relationships. It helps in understanding how objects interact and exist during system execution.

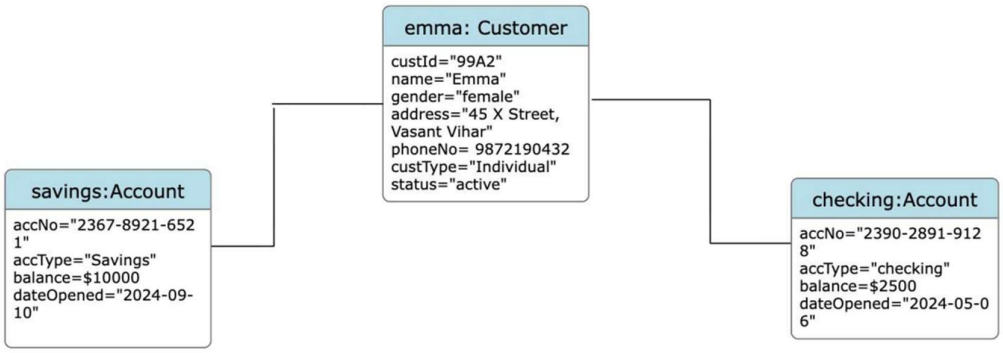
Object diagrams are especially useful for explaining complex class diagrams, validating system design, and visualizing real-time data scenarios. They are commonly used during the design and testing phases of the Software Development Life Cycle (SDLC) to verify whether the system structure behaves as expected.

Notations Used in Object Diagram

- **Object**: Represented by a rectangle containing the object name and class name (e.g., book1 : Book), underlined to indicate an instance.
- **Attributes**: Shown with specific values assigned to the object at that time.
- **Links**: Lines connecting objects, representing relationships between instances.
- **Multiplicity**: Usually implied by the number of objects rather than explicitly shown.
- **Visibility**: Can be indicated using + (public), - (private), or # (protected), though it is optional.

Ishaan Bhardwaj, A2305223339

UML OBJECT DIAGRAM:



Ishaan Bhardwaj, A2305223339

PRACTICAL 6

AIM: Consider a university database that keeps track of student and their majors, transcripts and registration and the university's courses. Several sections of each course are offered and each section is related to the instructor who is teaching. It also keeps track of the sponsored research projects of faculty and graduate students of the academic departments of the particular college. The database also keeps track of research grants and contracts awarded to the university. A grant is related to one principal investigator and to all researchers it supports.

THEORY:

An **Activity Diagram** is a type of **behavioral diagram** in the Unified Modeling Language (UML) that represents the flow of activities or actions carried out within a system. It is mainly used to model **workflows**, **business processes**, and **control flow** from one activity to another. Activity diagrams focus on the dynamic behavior of a system by showing how tasks are performed in sequence or in parallel.

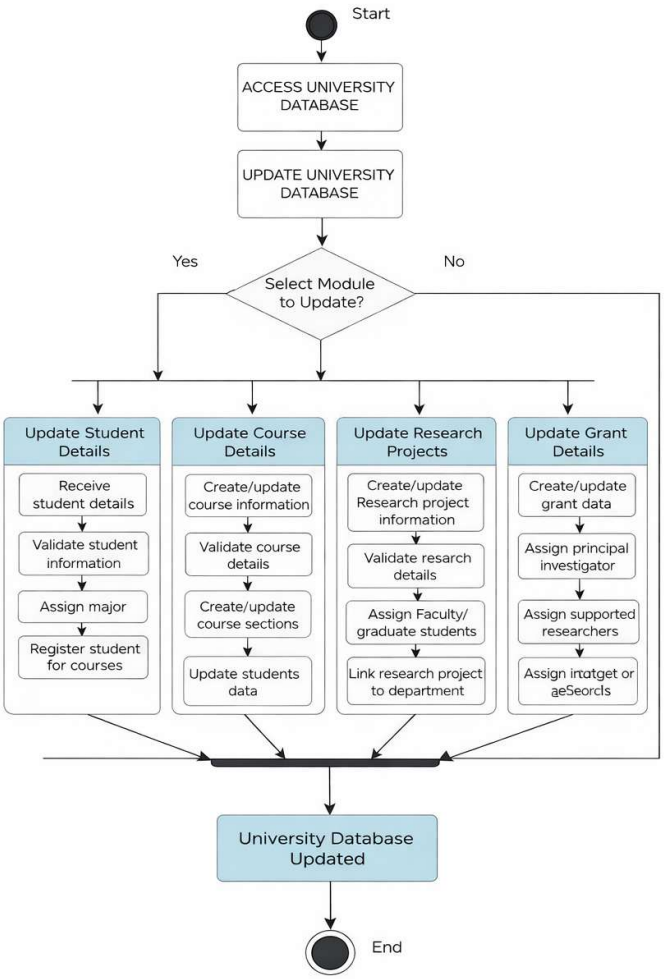
Activity diagrams are particularly useful during the **analysis and design phases** of the Software Development Life Cycle (SDLC). They help in understanding the overall logic of a process, identifying decision points, and visualizing the interaction between different activities involved in a system.

Notations Used in Activity Diagram

- **Initial Node**: Represented by a filled circle; indicates the starting point of the activity.
- **Activity / Action**: Shown as a rounded rectangle; represents a specific operation or task.
- **Control Flow**: Arrows showing the sequence of activities.
- **Decision Node**: Represented by a diamond; shows branching based on conditions.
- **Merge Node**: Combines multiple flows into a single path.
- **Fork Node**: Splits a flow into parallel activities.
- **Join Node**: Synchronizes parallel activities into one flow.
- **Final Node**: Represented by a filled circle with an outer ring; marks the end of the activity.

Ishaan Bhardwaj, A2305223339

ACTIVITY DIAGRAM:



PRACTICAL 7

AIM: To study about the sequence diagram in UML and Design the Sequence diagram for given scenario:

A customer places an order on an e-commerce website. The system verifies product availability, calculates the total cost, processes payment, generates an order ID, updates inventory, and sends order confirmation to the customer.

THEORY:

A Sequence Diagram is a type of UML (Unified Modeling Language) diagram that is used to represent the interaction between different objects or participants in a system in a time-ordered sequence. It shows how messages are exchanged between objects to perform a particular function or scenario. The main focus of a sequence diagram is on the order of communication, meaning it clearly shows which message is sent first, second, and so on. Time in a sequence diagram flows from top to bottom, while objects are arranged from left to right. Each object has a lifeline that represents its existence during the interaction, and messages are represented by arrows connecting these lifelines. Sequence diagrams are mainly used in the design phase of software development to understand system behavior, clarify requirements, and visualize how different components work together.

Notations of Sequence Diagram:

ACTOR:

An Actor represents an external entity that interacts with the system. It may be a human user, another system, or a hardware device. In a sequence diagram, the actor is drawn as a stick figure placed at the top of the diagram.

LIFELINE:

A Lifeline represents a participant or object involved in the interaction. It is drawn as a rectangle containing the object's name at the top, with a vertical dashed line extending downward. The dashed line indicates the existence of the object over time. Time in sequence diagrams moves from top to bottom.

ACTIVATION:

An Activation (also called Execution Specification) represents the time during which an object is performing an action or executing a method. It is shown as a thin vertical rectangle placed on a lifeline. The length of the rectangle indicates how long the object remains active. When an object receives a message, its activation bar starts, and when it completes the task, the activation ends.

SYNCHORONOUS MESSAGE:

A Synchronous Message represents a call where the sender waits for the receiver to complete the task before continuing. It is drawn as a solid line with a filled arrowhead pointing to the receiving object. This type of message behaves like a normal function call in programming.

ASYNCHORONOUS MESSAGE:

An Asynchronous Message is a message where the sender does not wait for the receiver to complete processing. It is drawn as a solid line with an open arrowhead. After sending the message, the sender continues its execution without blocking.

RETURN MESSAGE:

A Return Message represents the reply or result returned from the receiver back to the sender. It is shown as a dashed line with an open arrowhead. Although return messages are optional in sequence diagrams, they help in clearly showing the output of a method call.

SELF MESSAGE:

A Self Message occurs when an object sends a message to itself. It is drawn as a U-shaped arrow that starts and ends on the same lifeline. This usually represents a recursive function or an internal method call.

CREATION MESSAGE:

A Creation Message represents the creation of a new object during interaction. It is drawn as an arrow pointing to the object, and the lifeline of that object begins at that point. The object did not exist before this message.

DESTRUCTION MEAASGE:

A Destruction Message represents the termination of an object. It is shown by placing an “X” at the end of the object’s lifeline.

COMBINED FRAGEMENTS:

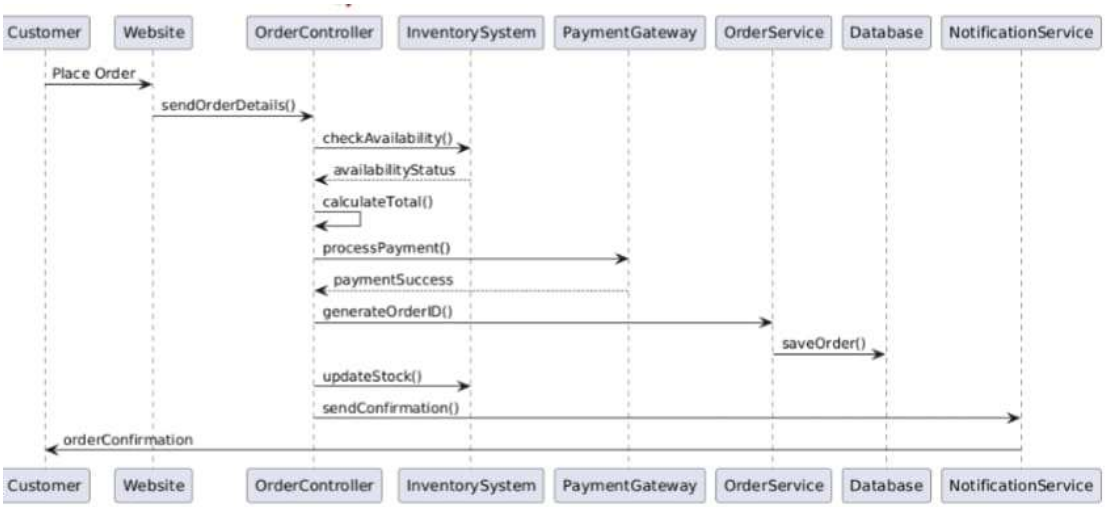
Combined Fragments are used to represent control structures like conditions and loops. They are drawn as large rectangular frames with a label in the top left corner such as alt (alternative), opt (optional), or loop.

GUARD CONDITION:

A Guard Condition is written inside square brackets [condition] and placed near a message or fragment.

Ishaan Bhardwaj, A2305223339

SEQUENCE DIAGRAM:



Ishaan Bhardwaj, A2305223339

PRACTICAL 8

AIM:

Study about collaboration diagram in UML and draw the collaboration diagram for the given scenario:

Many word processors, graphical editors and similar tools provide cut, copy and paste operations by means of some kind of a clipboard facility. At run-time suppose that such a system consists of an instance of an 'Editor' class linked to a number of instances of an 'Element' class. Elements are the items manipulated by a tool, such as word or shapes. Some of the elements may have been selected by the user. The editor is also linked to an instance of a 'Clipboard' class, which in turn maintains links to all the elements that have been placed on the clipboard. An element cannot simultaneously be displayed by the editor and on the clipboard.

THEORY:

A Collaboration Diagram in UML (Unified Modeling Language) is a type of interaction diagram that represents how objects communicate and work together to accomplish a particular task or behavior within a system. It focuses mainly on the structural organization of objects, their relationships, and the messages exchanged between them during the execution of a process.

In a collaboration diagram, different objects of classes are connected through links, and these links represent the relationships that allow objects to communicate. The diagram illustrates how objects collaborate with each other by sending and receiving messages to perform a specific functionality in the system.

Collaboration diagrams help developers and system designers understand how responsibilities are distributed among objects, how objects interact during system operation, and how different components cooperate to achieve system functionality. They provide a clear visualization of object collaboration in object-oriented systems.

These diagrams are widely used during the system design and analysis phase to model the dynamic behavior of software systems. They assist in identifying how objects are connected, how they exchange information, and how tasks are coordinated among them.

Collaboration diagrams are commonly used in:

- Object-Oriented System Design to represent interactions among objects.
- Modeling the dynamic behavior of systems during execution.

- Showing communication between objects in a structured manner.
- Understanding message passing and responsibilities of system components.
- Visualizing cooperation among classes and objects in complex software systems.

In a collaboration diagram, objects are represented as nodes, links represent associations between objects, and messages are represented by arrows along the links. Each message is labeled with a sequence number, which indicates the order in which interactions occur during the execution of a process.

Notations Used in Collaboration Diagram:

1. Object

An object represents an instance of a class that participates in the interaction.

Notation:

- Represented by a rectangle
- Format: objectName : ClassName

2. Link (Association)

A link represents the connection between two objects that allows them to send messages to each other.

Notation: Drawn as a solid line connecting objects.

3. Message

A message represents communication between objects, usually invoking a method or operation.

Notation:

- Represented by an arrow on the link
- Includes a sequence number

4. Sequence Number

Sequence numbers indicate the order of messages sent between objects.

5. Role Name

Sometimes a link includes a role name to show the role played by the object in the relationship.

6. Self Message

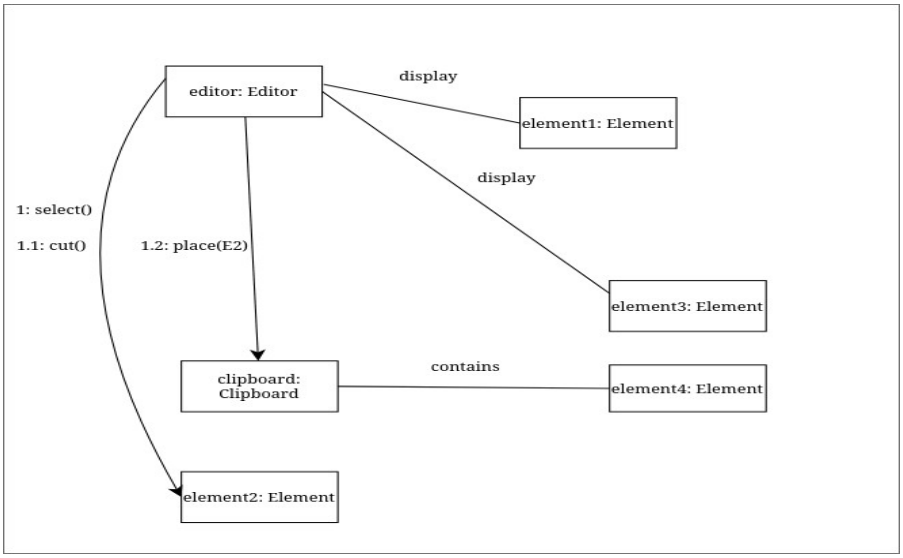
An object can send a message to itself.

7. Multiplicity

Shows how many objects can participate in the relationship.

Ishaan Bhardwaj, A2305223339

COLLABORATION DIAGRAM:



Ishaan Bhardwaj, A2305223339

PRACTICAL 9

AIM:

Study about deployment diagram in UML and draw deployment diagram for E-Commerce website deployment.

THEORY:

Deployment Diagram in UML (Unified Modeling Language) is used to represent the physical architecture of a system. It shows how software components are deployed on hardware nodes and how these nodes communicate with each other in a network.

Deployment diagrams are mainly used during the system design phase to illustrate the physical distribution of software components, such as servers, databases, devices, and networks.

This diagram helps developers and system architects understand where software components are installed and how the system will run in the real environment.

<u>Purpose of Deployment Diagram:</u>

The main purposes of a deployment diagram are:

- To represent the physical layout of hardware devices in a system.
- To show how software artifacts are deployed on hardware nodes.
- To describe communication between nodes.
- To visualize the runtime architecture of the system.
- To help in system configuration and infrastructure planning.

Deployment diagrams are widely used in systems such as web applications, distributed systems, cloud-based systems, and enterprise applications.

<u>Notations Used in Deployment Diagram:</u>

1. Node

A Node represents a physical device or computational resource that executes software.

Examples:

- Application Server
- Database Server

Notation:
A node is represented by a 3D box.

2. Artifact

An Artifact is a physical piece of information or software that is deployed on a node.

Examples:

- Executable files (.exe)
- Web application (.war)

Notation:
Artifact is represented as a rectangle with a folded corner.

3. Communication Path

A Communication Path shows the network connection between nodes that allows them to communicate.

Notation:
A solid line connecting nodes.

4. Component

Sometimes deployment diagrams may include software components to represent logical parts of the system deployed as artifacts.

Examples:

- Payment Module
- Authentication Service
- Product Service

Notation:
A rectangle with two small rectangles on the side.

5. Interface

An Interface defines the services provided or required by components.

6. Dependency

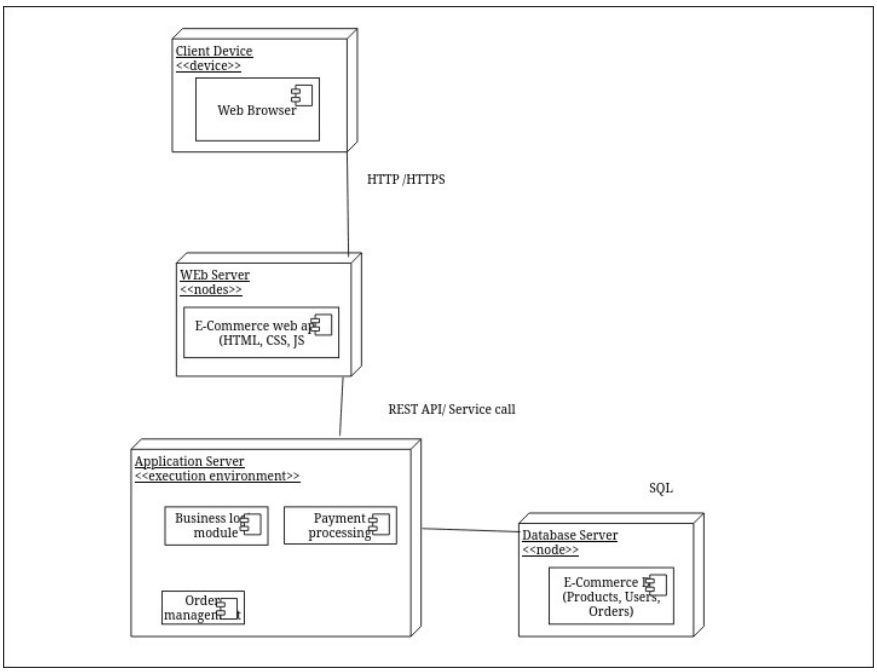
A Dependency indicates that one element depends on another element. It is represented by dashed arrow.

7. Deployment Relationship

A Deployment Relationship shows that an artifact is deployed on a node.

Ishaan Bhardwaj, A2305223339

Advantages of Deployment Diagram

- Helps visualize system infrastructure
- Shows hardware–software mapping
- Useful for distributed and cloud systems
- Helps in performance and scalability planning
- Improves system understanding for developers and stakeholders

DEPLOYMENT DIAGRAM: